Orchestration

These scripts run the sample pipeline in ordered steps:

1. extract per-sample DuckDB files from BAM indexes
2. apply the dbt models to each per-sample DuckDB file
3. merge all per-sample DuckDB files into one combined DuckDB file
4. run merged-file dbt models on the combined DuckDB file
5. generate per-sample plots from the merged DuckDB file
6. train the logistic regression model on the merged DuckDB file

Required Dependencies

- `uv`
- `jq`

The project dependencies for `bam_processing` and `dbt_models` are installed and run through `uv`.

Required Environment Variables

- `BAM_DIR` Directory containing the `.bai` files referenced by `config/sample_config.json`.
- `FASTA_PATH` Reference FASTA used by `bam_processing` during feature extraction.
- `OUTPUT_DIR` Base output directory. DuckDB files are written under `$OUTPUT_DIR/duckdb`.

Optional:

- `CHUNK_SIZE` Maximum number of source BAM records processed per extractor chunk. Default: `10000`
- `JAX_PLATFORMS` JAX backend used by `bam_processing`. Use `cpu` by default, or `METAL` on Apple Silicon if `jax-metal` is installed.
- `MERGED_BAM_PATH` Output path for the merged BAM file created by the optional BAM merge step. Default: `$BAM_DIR/all_samples.bam`
- `MERGED_DUCKDB_PATH` Output path for the merged DuckDB file created by step 3. Default: `$OUTPUT_DIR/duckdb/all_samples.features.duckdb`

Run

From the repo root:

```
export BAM_DIR=/path/to/bam_indexes
export FASTA_PATH=/path/to/reference.fa
export JAX_PLATFORMS=cpu
export OUTPUT_DIR=/tmp/prefect
```

Run the scripts in order:

```
bash orchestration/00_run_all.sh
```

Or run them one by one:

```
bash orchestration/01_run_bam_processing.sh
bash orchestration/02_run_dbt_models.sh
bash orchestration/03_merge_duckdb_files.sh
bash orchestration/04_run_merged_dbt_models.sh
bash orchestration/05_run_plotting.sh
bash orchestration/06_run_model.sh
```

What Each Script Does

`00_run_all.sh`

- runs the full pipeline in sequence from `01` through `06`

`01_run_bam_processing.sh`

- reads `config/sample_config.json` with `jq`
- runs `uv run --project bam_processing bam-processing ...` for each sample
- forwards `CHUNK_SIZE` to `--chunk-size`
- writes one DuckDB file per sample to `$OUTPUT_DIR/duckdb/<sample>.features.duckdb`

`02_run_dbt_models.sh`

- reads `config/sample_config.json` with `jq`
- runs `dbt run --select tag:individual` for each per-sample DuckDB file
- sets `DBT_DUCKDB_PATH` and `DBT_DUCKDB_DATABASE` per sample before running dbt

`03_merge_duckdb_files.sh`

- runs `uv run --project dbt_models python dbt_models/merge_duckdb_files.py`
- merges all `*.features.duckdb` files in `$OUTPUT_DIR/duckdb`
- writes the merged DuckDB file to `$MERGED_DUCKDB_PATH` or the default output path

`04_run_merged_dbt_models.sh`

- runs `dbt run --select tag:merged` on the merged DuckDB file
- reads the merged file from `$MERGED_DUCKDB_PATH` or `$OUTPUT_DIR/duckdb/all_samples.features.duck`

`05_run_plotting.sh`

- runs the standalone plotting scripts in `plotting/`
- reads the merged file from `$MERGED_DUCKDB_PATH` or `$OUTPUT_DIR/duckdb/all_samples.features.duck`
- writes one PDF per sample per plot type into `output/plots`

`06_run_model.sh`

- runs the logistic regression training script in `models/`
- reads the merged file from `$MERGED_DUCKDB_PATH` or `$OUTPUT_DIR/duckdb/all_samples.features.duck`

- writes model metrics and the ROC curve into `output/models`